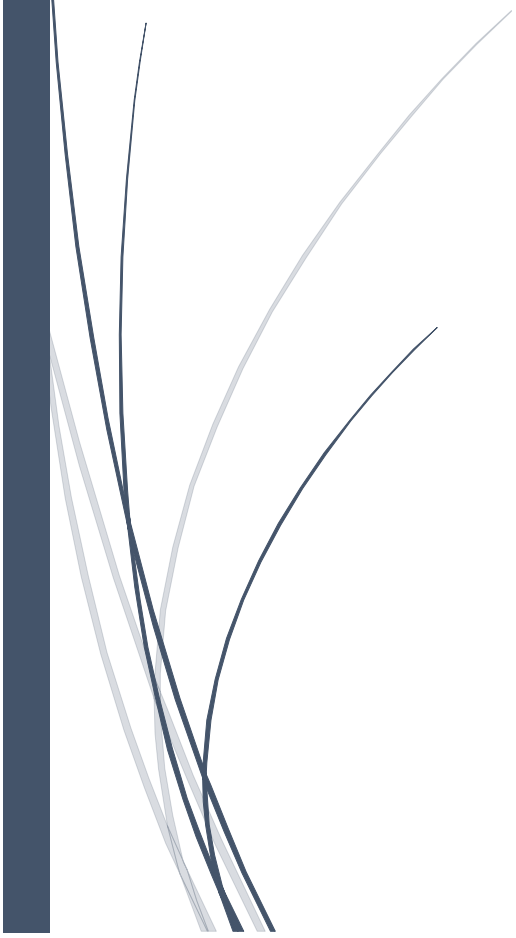
@VANZA

# UT3. Fundamentos del Lenguaje

CFGS

Desarrollo de Aplicaciones Multiplataforma



Dpto. Informática
IES AUGUSTÓBRIGA (NAVALMORAL DE LA MATA)

Contenido

1. Introducción. ....2
2. Asignaciones simples y múltiples. ....2
2.1. Asignaciones Simples ....2
2.2. Asignaciones de múltiples variables. ....2
2.3. Asignaciones Múltiples. ....3
2.4. Asignaciones Slicing con * ....4
3. Control de flujo de ejecución. ....5
3.1. Control de flujo condicional con las sentencias if, elif, else. ....5
3.2. Implementaciones de switch case en Python. ....6
3.3. Sentencia if ternaria. ....7
4. Flujo de ejecución con Bucles ....7
4.1. Bucle while ....7
4.2. Bucle for ....9
4.3. Control de flujo dentro de bucles: break y continue ....10
5. Operador Walrus para asignar ....11
6. Funciones en Python ....12
6.1. Ejemplo de modularización de código. ....13
6.2. Parámetros y argumentos de funciones ....16
6.3. Uso de args y kwargs ....17
6.4. Funciones recursivas ....19
6.5. Funciones anónimas: Expresiones Lambda ....23
6.6. Funciones de orden superior ....25
6.7. Funciones dentro de funciones ....26
6.8. Espacios de nombres y contextos ....27
6.9. Decoradores ....29
6.10. Documentación de funciones ....31
6.11. Funciones generadoras ....32
7. Excepciones ....34
7.1. Excepciones conocidas ....36
7.2. Elevar excepciones de forma manual. ....38
7.3. Definición de excepciones propias ....38

# 1. Introducción.

En esta unidad se verán conceptos fundamentales para la programación en Python, como las estructuras de control, las asignaciones, las funciones y las excepciones, de manera que podamos definir la lógica de un programa en Python.

En capítulos anteriores ya se ha visto la asignación (=). Veremos las estructuras de control de flujo lógico de algoritmos y programas en Python, definiciones formales, ejemplos, consejos y buenas prácticas de uso.

Las funciones son la **forma mínima de encapsular trozos de código** y permiten utilizar ese código en diferentes partes de la lógica de los programas.

Las excepciones juegan un papel fundamental en el desarrollo de software, permitiendo detectar situaciones anómalas y construir programas resilientes a errores controlando las casuísticas que ocurren en la ejecución de los programas.

# 2. Asignaciones simples y múltiples.

A pesar de su aparente simplicidad, las asignaciones de objetos esconden muchos detalles de los que se puede sacar mucho partido si se usan de forma correcta.

## 2.1. Asignaciones Simples

Son las más utilizadas.

```
>>> a = 1
>>> b = 'Arbol'
>>> c = map(lambda x: x / 5, range(50))
```

Se componen de una sola variable en la parte izquierda y de un valor o un literal en la parte derecha de la asignación, separando las partes con un '='.

## 2.2. Asignaciones de múltiples variables.

Son asignaciones algo más complejas y hay que verlas con cuidado porque pueden dar lugar a confusión. Se componen de varias asignaciones encadenadas que de izquierda a derecha se van haciendo asignaciones de un mismo objeto a variables diferentes. El valor que se asigna es el que está más a la derecha y se irá asignando a todas las variables que estén a la izquierda.

Dependiendo de si el valor es mutable o inmutable, el cambio de una de las variables inicializadas de este modo puede afectar a las demás:

```
>>> a=b=c=1
>>> a,b,c
(1, 1, 1)
>>> b=2
>>> a,b,c
(1, 2, 1)
```

Como ya conocemos, las variables de tipo 'int' son inmutables de manera que los cambios en alguna de las variables no afecta a las demás, como puede verse.

Otro ejemplo con listas:

```
>>> d=e=[1,2,3]
>>> e.append(4)
>>> d,e
([1, 2, 3, 4], [1, 2, 3, 4])
```

En este ejemplo, asignamos una lista, que es un elemento mutable a 'd' y 'e'. Al modificar 'e', el cambio se realiza también en 'd'

Debemos tener mucho cuidado con las modificaciones sobre objetos mutables dado que podemos tener comportamientos no deseados.

### 2.3. Asignaciones Múltiples.

Se basan en asignar valores a varias variables a la vez en una sola sentencia de código.

```
>>> a,b,c=[1,2,3]
>>> a,b,c
(1, 2, 3)
>>> a
1
>>> b
2
>>> c
3
```

Otro ejemplo:

```
>>> d,e,f='Hola'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: too many values to unpack (expected 3)
```

Como puedes comprobar, nos da error debido a que estamos declarando tres variables y la cadena 'Hola' tiene 4 caracteres, de forma que tenemos más valores que variables.

```
>>> d,e,f='mar'
>>> d,e,f
('m', 'a', 'r')
```

Cada elemento de la secuencia de la derecha se desempaqueta uno a uno por lo que es necesario que el número de variables sea igual al número de elementos por asignar.

La asignación de variables se hace de izquierda a derecha por lo que debemos tener cuidado con algunas situaciones que se pueden producir:

```
>>> a=[1,2,3]
>>> i=0
>>> i,a[i]=2,245
>>> a
[1, 2, 245]
```

Como podemos ver, el valor actualizado de 'a' es el que ocupa la posición 2 dado que en la misma sentencia se ha producido con anterioridad la modificación del índice 'i'.

### 2.4. Asignaciones Slicing con *

Existe una forma más sofisticada de hacer asignaciones cuando se trata de secuencias y que consiste en hacer uso del operador '*', que se utiliza para descomponer una secuencia en cada uno de sus elementos:

```
>>> lista=[1,2,3,4,5,6]
>>> a, *b, c=lista
>>> a
1
>>> b
[2, 3, 4, 5]
>>> c
6
```

```
>>> d,e,f=lista[0], lista[1:-1], lista[-1]
>>> d
1
>>> e
[2, 3, 4, 5]
>>> f
6
```

Como se puede comprobar, se puede hacer slicing (código de la derecha) de una forma más simple con el operador '*' (código de la izquierda), de manera que podamos descomponer una lista (o cualquier secuencia) en su primer elemento, los siguientes y el último.

## 3. Control de flujo de ejecución.

En los lenguajes de programación, al desarrollar programas o algoritmos, se utilizan sentencias que permiten encauzar el flujo de la ejecución de un programa, de manera que se puede especificar si se deben ejecutar unas operaciones u otras en función de unas expresiones o funciones lógicas.

### 3.1. Control de flujo condicional con las sentencias if, elif, else.

Para controlar la ejecución de forma condicional se utilizan las sentencias if, elif y else. Se suelen utilizar a modo de semáforo lógico en el que, dependiendo de las expresiones usadas y de sus valores, el flujo de ejecución seguirá un camino u otro.

"if" expresión ":" #bloque de código

("elif" expresión ":" #bloque de código) *

["else" ":" #bloque de código]

"elif" y "else" son opcionales y además, "elif" puede aparecer un número indeterminado de veces.

Ejemplo:

pruebas.py > ...

```
1   #permite la introducción de un valor por consola
2   #y lo transforma en un entero
3   numero=int(input("Introduce un número: "))
4   #comparamos "si" el número es menor que 0
5   if numero < 0 :
6       print('El número es negativo.')
7   #si no es menor que 0, comparamos si es igual a 0
8   elif numero == 0:
9       print('El numero es cero.')
10  #si no es menor que 0 y tampoco es igual a 0...
11  else:
12      print('El número es positivo')
13
```

Como se puede comprobar en el ejemplo anterior, para comenzar a controlar el flujo se utiliza la cláusula "**if"** seguida de una expresión. Si se pretende comprobar otra expresión cuando la anterior o anteriores ha resultado "falsa", se utilizan tantas clausulas **elif** como sean necesarias y al final de la comprobación se añade opcionalmente una cláusula **else**, que el programa seguirá ejecutando si todas las anteriores expresiones se evaluaron como falsas.

### 3.2. Implementaciones de switch case en Python.

Un control de flujo muy utilizado en cualquier lenguaje de programación es el **switch (**o **case** dependiendo del lenguaje**)** con el que se pretende encauzar la ejecución del código dependiendo del valor de una variable utilizada como operador.

Hasta la versión 3.10 de Python, no existía una estructura de control de flujo igual en Python pero era fácil de simular con las sentencias **if, elif, else.**

Desde la citada versión 3.10, Python cuenta con una estructura muy similar:

```
match "subject":
    case "patron_1":
        acciones
    case "patron_2":
        acciones
    case "patron_3":
        acciones
    …
    case _:
        acciones_en otro caso
```

Donde **match** y **case** son palabras reservadas, "subject" es la variable que será utilizada para comparar, patron_x son los patrones a evaluar si se cumplen y acciones con las sentencias que se ejecutarán si se cumple. Por defecto se puede usar "_" que se ejecutará si ninguno de los patrones anteriores coincide.

Ejemplo:

pruebas.py > ...

```python
comando=input('Introduce el comando (iniciar, detener, reiniciar, ayuda): ')
match comando:
        case "iniciar":
            print("El sistema se está iniciando...")
        case "detener":
            print("El sistema se está deteniendo...")
        case "reiniciar":
            print("El sistema se está reiniciando...")
        case "ayuda":
            print("Lista de comandos disponibles: iniciar, detener, reiniciar, ayuda")
        case _:
            print("Comando no reconocido")
```

### 3.3. Sentencia if ternaria.

En Python se puede simplificar la sentencia **if** mediante una única sentencia, denominada **ternaria** que se utiliza solamente en los casos en los que está clara la lógica expresada.

Ejemplo:

pruebas.py > ...

```python
edad=50
categoria = "Cadete" if edad < 15 else "Adulto"
print(categoria)
```

## 4. Flujo de ejecución con Bucles

A menudo, el flujo de un algoritmo se mantiene realizando operaciones hasta que una expresión lógica cambie de valor u ocurra algún acontecimiento esperado. Este tipo de ejecuciones se implementan mediante bucles, que son trozos de código que deben ser ejecutados hasta que una expresión determinada ocurra.

### 4.1. Bucle while

Permite permanecer iterando sobre un bloque del código continuamente “mientras” una expresión sea verdadera.

Opcionalmente se puede añadir una expresión **else**, que se ejecutará siempre que la primera expresión no se evalúe como verdadera.

**while** "condición" :

acciones

**else:**

acciones

Ejemplo:

pruebas.py > ...

```
1   i = 0
2   while i < 5:
3       print(f'Iteración {i}')
4       i +=1
5   else:
6       print(f'la variable i ya es igual que 5')
```

Salida por pantalla:

```
Iteración 0
Iteración 1
Iteración 2
Iteración 3
Iteración 4
la variable i ya es igual que 5
```

Estos bucles pueden generar bucles infinitos, que se usan de forma controlada cuando se pretende ejecutar constantemente un código a la espera de un evento específico.

Veamos el siguiente ejemplo que lee la información proporcionada por el usuario hasta que el usuario introduce un 0:

pruebas.py > ...

```
1   enBucle = True
2   while enBucle:
3       x = int(input('Introduce un número:'))
4       if x > 0:
5           print(f'El número {x} es positivo')
6       elif x < 0:
7           print(f'El número {x} es negativo')
8       elif x == 0:
9           enBucle = False
10  else:
11      print('Ejecución terminada')
```

Salida por pantalla:

```
Introduce un número:4
El número 4 es positivo
Introduce un número:5
El número 5 es positivo
Introduce un número:8
El número 8 es positivo
Introduce un número:0
Ejecución terminada
```

El programa se ejecuta en bucle mientras no se introduce el número 0. En ese momento, la variable de control “enBucle” cambia a “False” y deja de cumplirse la condición por lo que la ejecución pasa a la sentencia “else”.

### 4.2. **Bucle** for

Otra forma de construir bucles es mediante la sentencia **for.** Sus usos más notables son tanto para controlar el flujo de ejecución como para iterar por iterables.

Opcionalmente se puede añadir una sentencia **else**, la cual se ejecutará al terminar la iteración

**for** “iterador” **in** “iterable”:

    acciones

**else:**

    acciones

Ejemplo:

pruebas.py > ...

```
1   for x in range(10):
2       if x >= 2 and not x % 2:
3           print (x)
4   else:
5       print('Fin del interador')
```

Salida por pantalla:

```
2
4
6
8
Fin del interador
```

El código anterior recorre los números comprendidos entre 0 y 10 (excluyendo el 10) y devuelve los números que son pares. Cuando finaliza el recorrido del iterador, ejecuta la sentencia del **else.**

Veamos un ejemplo:

pruebas.py > ...

```
1   for nomEquipo, ptos in [('Equipo1',89),('Equipo2',90)]:
2       print(f'{nomEquipo}:{ptos}')
3
```

Salida por pantalla:

```
Equipo1:89
Equipo2:90
```

### 4.3. Control de flujo dentro de bucles: break y continue

En los bucles se puede cambiar el curso de las iteraciones y hacer que algunas se salten o directamente que se pare la ejecución. Para este fin existen las sentencias **continue** y **break.**

Cuando se pretende detener la ejecución del bucle se usa la sentencia **break:**

pruebas.py > ...

```
1   i = 0
2   while i < 50:
3       print(i)
4       i += 1
5       if i > 3:
6           break
```

Es muy sencillo. A pesar de que la condición del bucle **while** es que la variable sea menor que 50 (lo que pretende que se itere hasta el número 49), la condición de que la variable "i" sea mayor de 3 provocará la llamada a la sentencia "break" y la salida del bucle inmediatamente.

Salida por pantalla:

```
0
1
2
3
```

Otro ejemplo:

pruebas.py > ...

```
1   for i, name in enumerate(['Pepe','Maria','Juan','Mario','Paloma']):
2       print(f'{i}: {name}')
3       if i > 1:
4           break
```

Observa que en el ejemplo anterior, se abandona el bucle cuando la variable i=2 pero ya ha sacado por pantalla el índice 2 y su valor.

Salida por pantalla:

```
0: Pepe
1: Maria
2: Juan
```

Si lo que se pretende es omitir la ejecución en curso y seguir con la siguiente volviendo a evaluar la expresión del bucle, se utiliza la sentencia **continue**.

pruebas.py > ...

```
1   for name in ['Pepe', 'Paloma', 'María', 'Manuel', 'Emilio', 'José', 'Antonia']:
2       if len(name) < 5:
3           continue #se omite la ejecución cuando el nombre tiene menos de 5 caracteres
4       print(name)
5
```

Salida por pantalla:

```
Paloma
María
Manuel
Emilio
Antonia
```

## 5. Operador Walrus para asignar

Desde la versión 3.8 de Python se cuenta con el operador ":=", denominado walrus

Nombre_variable := expresión

Se utiliza para asignar variables mientras estas son utilizadas en expresiones, simplificando el código y mejorando la legibilidad. Lo vemos con un ejemplo:

Sin walrus

pruebas.py > ...

```
1   n=range(5)
2   len_n=len(n)
3   if len_n > 2:
4       print(f'{len_n} es mayor que 2')
```

Con walrus

pruebas.py > ...

```
1   n=range(5)
2   if (len_n := len(n)) > 2:
3       print(f'{len_n} es mayor que 2')
```

## 6. Funciones en Python

La función se define como un subprograma o subrutina que forma parte de un algoritmo principal. Aunque en algunos lenguajes de programación se distinguen aquellos subprogramas que devuelven valores (función) de los que no (procedimientos), en Python, todos devuelven **None** por defecto, de manera que todos se consideran **funciones.**

El objetivo principal de las funciones es ayudar a organizar mejor el código, mejorar la legibilidad y reutilizar lógica común en diferentes partes del mismo.

**def** "nombre_función" **(parám1, param2…):**

acciones

**return**

**def**: Indica el inicio de la declaración de la función.
**nombre_función**: Identificador que sigue las reglas de nomenclatura en Python (letras, números y guion bajo _, sin espacios, y no puede comenzar con un número ni coincidir con palabras reservadas).
**([parámetros])**: Los paréntesis son obligatorios. Contienen los parámetros (opcionales), separados por comas.
**return** (opcional): Permite devolver un valor o resultado al final de la ejecución de la función.

Algunos ejemplos:

```
1  def imprimir():
2      pass #es un marcador que no hace nada y permite que la función
3          # que en este caso no hace nada, no lance un error.
4
```

Esta función no hace nada.

```
1  def imprimir(cadena):
2      print(f'{cadena}')
3
4  imprimir('Hola Mundo')
5
```

Recibe por parámetro un valor que se pasa a la función "print" para sacar por pantalla.

Llamada a la función pasándole la cadena 'Hola Mundo'

```
1  def imprimir():
2      return ('Hola Mundo')
3
4  print(imprimir())
```

Se define la función que devuelve la cadena 'Hola Mundo' mediante **return.**

Llamamos a la función "print" pasándole la función "imprimir"

Más ejemplos y sus resultados:

```
1   def menorValor(iter):
2       return min(iter)
3
4   salida = menorValor(range(5))
5   print(salida)
```

Se define la función con un parámetro iterable que devuelve el valor mínimo

Declaramos una variable que recoge el valor devuelto por la función y la muestra por pantalla

Salida: 1

```
1   def iniciales(cadena,sep=' '):
2       #Esta función devuelve las iniciales de
3       #cada palabra en la cadena separadas por
4       # "sep" que en este caso es espacio en blanco
5       return ''.join(x[0].upper() for x in cadena.split(sep))
6
7   print(iniciales('Antonio García Fernández'))
```

Salida: AGF

Si realizamos la llamada a:

```
print(iniciales('el,árbol,del,parque',sep=','))
```

Y le pasamos como separador ‘,’ la salida será: EÁDP

Con estos pequeños ejemplos queda patente la gran utilidad que tiene el uso de las funciones para que con el mismo código se puedan resolver problemas distintos.

### 6.1. Ejemplo de modularización de código.

Aunque en los siguientes ejemplos aparecen expresiones y elementos que no hemos visto aún, vamos a utilizarlos para entender perfectamente el concepto de modularidad y cómo la modularización de código nos va a permitir, entre otras cosas, realizar cambios en una parte de la lógica del programa sin que afecte al programa en general.

Partimos del siguiente programa: “*Supongamos que partiendo de una cadena de caracteres, se pretende separar la cadena mediante un separador y aplicar una función basada en el algoritmo de cifrado md5 a cada elemento para obtener su forma hexadecimal. Una vez obtenido el cifrado, se ha de guardar en un fichero de texto plano.*

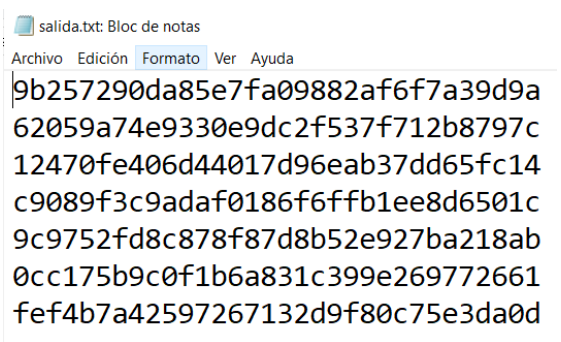
pruebas.py > ...

```python
import hashlib
"""La siguiente función recibe por parámetro
una cadena y en función del separador que le pasemos
devuelve cada elemento de esa cadena en una lista"""
def extraerIdentificadores(cadena_original, sep=' '):
    return cadena_original.split(sep)
"""La siguiente función recibe una lista de elementos
y aplica a cada elemento un cifrado md5 para obtener
su forma hexadecimal, devolviendo finalmente una
lista con los elementos cifrados"""
def aplicarHash(lista):
    md5s = []
    for cad in lista:
        m = hashlib.md5(cad.encode('utf-8'))
        md5s.append(m.hexdigest())
    return md5s

"""La siguiente función escribe en un fichero pasado
por parámetro los datos pasados igualmente por parámetro"""
def escribir_a_fichero(data, nombreFichero='salida.txt'):
    with open(nombreFichero,'w') as f:
        for d in data:
            f.write(d + '\n')
        return nombreFichero
"""Una vez definidas las funciones que vamos a necesitar,
hacemos las inicializaciones y llamadas a esas funciones"""

cadenaOriginal='Este<>texto<>es<>la<>cadena<>a<>guardar'
indentificadores=extraerIdentificadores(cadenaOriginal,'<>')
idModificados=aplicarHash(indentificadores)
#observa que en la siguiente llamada a la función
#no es necesario pasar el parámetro con el nombre del fichero
escribir_a_fichero(idModificados)
```

Y esta es la salida del fichero de texto una vez lanzado el programa…



Suponemos ahora que cambia la especificación del problema: *se pretende aplicar la función de cifrado sha256 en vez de md5 y guardar en un formato JSON en lugar de texto plano.*

pruebas.py > ...

```python
import hashlib
import json
"""La siguiente función recibe por parámetro
una cadena y en función del separador que le pasemos
devuelve cada elemento de esa cadena en una lista"""
def extraerIdentificadores(cadena_original, sep=' '):
    return cadena_original.split(sep)
"""La siguiente función recibe una lista de elementos
y aplica a cada elemento un cifrado sha256 para obtener
su forma hexadecimal, devolviendo finalmente una
lista con los elementos cifrados"""

def aplicarHash(lista):
    hashes = []
    for cad in lista:
        m = hashlib.sha256(cad.encode('utf-8'))
        hashes.append(m.hexdigest())
    return hashes

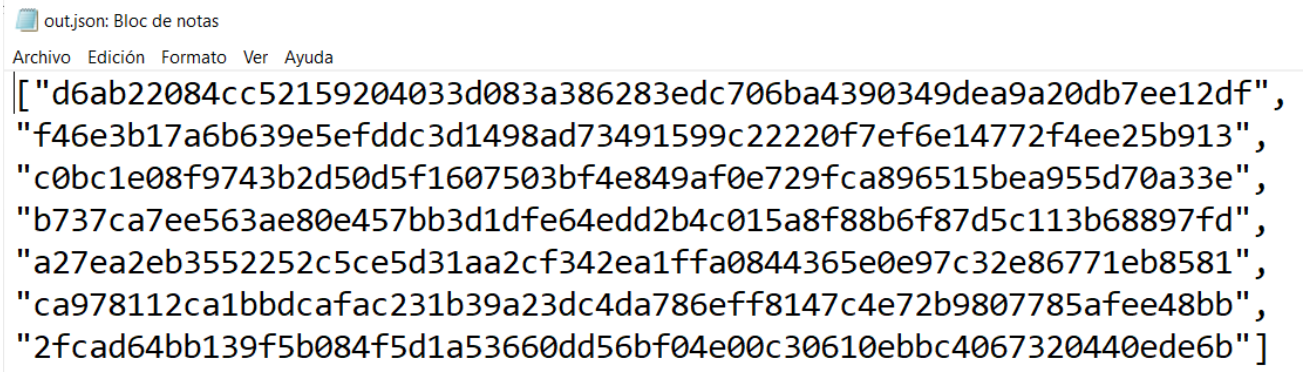
"""La siguiente función escribe en un fichero pasado
por parámetro los datos pasados igualmente por parámetro
pero a diferencia del ejemplo anterior, lo hace
sobre un fichero de tipo json por lo que
se utiliza la librería json"""

def escribir_a_json(data, nombre_fichero='out.json'):
    with open(nombre_fichero,'w') as f:
        json.dump(data,f)
    return nombre_fichero

"""Una vez definidas las funciones que vamos a necesitar,
hacemos las inicializaciones y llamadas a esas funciones"""

cadenaOriginal='Este<>texto<>es<>la<>cadena<>a<>guardar'
indentificadores=extraerIdentificadores(cadenaOriginal,'<>')
idModificados=aplicarHash(indentificadores)
#observa que en la siguiente llamada a la función
#no es necesario pasar el parámetro con el nombre del fichero
escribir_a_json(idModificados)
```

Y esta es la salida del fichero “json” una vez ejecutado el programa…




Como se puede comprobar, sólo cambiando las funciones necesarias se puede realizar una lógica completamente diferente sin que aplique al programa en general. Del mismo modo, se podría haber modificado alguna operación específica dentro de la función sin cambiar ninguna otra parte del código. Es aquí donde radica la potencia de modularización con funciones y su facilidad para encapsular código.

## 6.2. Parámetros y argumentos de funciones

Las funciones pueden utilizar variables que se introducen desde el exterior y que se conocen como **parámetros de la función.**

En Python los parámetros se definen como una sucesión de indentificadores que definen variables y que se separan por comas. Opcionalmente se puede definir un valor por defecto asignado mediante “=” y un tipo para cada dato utilizado mediante “:”. Incluso se puede definir el tipo de dato que devolverá la función mediante “-> tipo”

```
1   def division(num, deno=2):
2       return num / deno
3
4   def permu(num:int, veces:float)->float:
5       return num * veces
```

Todos los parámetros son pasados por referencia, lo que significa que sólo se copian los punteros de las variables y no el contenido en sí. La desventaja de esto y que hay que tener en cuenta es que si estamos tratando con objetos mutables y su contenido se modifica en el interior de la función, se modificará en la referencia de ese objeto fuera de la función.

```
def suma(objeto1, objeto2):
    objeto1+=objeto2
    return objeto1

a = [1,2,3,4]
b = [5,6]
c = suma(a,b)
print(c)
print(a)
```

Como se puede observar, al tratarse de listas, que son objetos mutables, el objeto “a” se ha modificado dentro de la función y por tanto, se ve modificado fuera.

Salida:

```
[1, 2, 3, 4, 5, 6]  ➔ c
[1, 2, 3, 4, 5, 6]  ➔ a
```

Cuando se utilizan valores por defecto en las funciones, hay que tener especial precaución de que estos no sean de tipo mutable. Si es así, se recomienda cambiarlos por **None** y comprobar si el valor como argumento en **None** para inicializar:

```
1  def añadir_valor_seguro(elem, lst=None):
2      if lst is None:
3          lst=[]
4      lst.append(elem)
5      return lst
```

## 6.3. Uso de args y kwargs

Como se ha comentado, los argumentos en las funciones se pueden pasar posicionalmente o usando clave-valor donde el nombre del parámetro es la clave y se le asigna el valor con “=”. Pues bien, en la definición de funciones se puede especificar cada grupo y cómo trabajar con ellos.

Para identificar los parámetros pasados a la función como argumentos posicionales, se descomponen usando un simple carácter ‘*’ (del mismo modo que para descomponer listas o cualquier secuencia) y para identificar o descomponer los parámetros pasados como clave-valor se utilizan dos caracteres ‘**’. A los argumentos posicionales se les denomina **args** y a los clave-valor **kwargs.** Veamos un ejemplo de su uso:

pruebas.py > ...

```python
def mi_funcion(*args, **kwargs):
    print(f'args:{args}, kwars:{kwargs}')

#llamamos a la función de distintas formas
#sin parámetros
mi_funcion()
#con un parámetro posicional
mi_funcion(42)
#con un parámetro clave-valor
mi_funcion(vehiculo='Coche')
#con los dos tipos de parámetros
mi_funcion(125,'Juan',modo='vuelo',flag='activado')
```

Y la salida por pantalla:

```
args:(), kwars:{}
args:(42,), kwars:{}
args:(), kwars:{'vehiculo': 'Coche'}
args:(125, 'Juan'), kwars:{'modo': 'vuelo', 'flag': 'activado'}
```

Como se puede comprobar, todos los parámetros que se pasan posicionalmente se guardan en **args** y los que se pasan como clave-valor se guardan en **kwargs** sin necesidad de añadir nada a la definición de la función. En Python, los parámetros de las funciones son estrictos y si no se define el número de parámetros con args y kwargs, las funciones elevarán excepciones al intentar ser ejecutadas con más parámetros de los definidos.

Los args y kwargs son muy útiles a la hora de extender clases o de ejecutar muchas funciones que hagan uso de parámetros que se provean solo en la llamada original:

pruebas.py > ...

```python
def calculo_general(**kwargs):
    ret_s=suma(**kwargs)
    ret_d=division(**kwargs)
    return ret_s * ret_d

def suma(op1,op2,**kwargs):
    return op1 + op2

def division(op3,op4,**kwargs):
    return op3 / op4

res=calculo_general(op1=3,op2=5,op3=8,op4=2)

print(res)
```

Observa que cuando se define **calculo_general** no se especifican parámetros sino que se pasan todos a las funciones internas (suma y división). En la definición de cada función, se definen por nombre los parámetros que se deben usar y el resto en **kwargs.

Al hacer la llamada a la función **calculo_general** se permite añadir tantos parámetros clave-valor que se desee sin que afecte al funcionamiento dado que únicamente se utilizarán los que se definen con nombre.

Del mismo modo se puede realizar con parámetros posicionales y *arg:

pruebas.py > ...

```
1  def calculo_general(*args):
2      ret_s=suma(*args)
3      ret_d=division(*args)
4      return ret_s * ret_d
5
6  def suma(op1,op2,*args):
7      return op1 + op2
8
9  def division(a1,a2,op3,op4):
10     return op3 / op4
11
12 res=calculo_general(3,5,8,2)
13
14 print(res)
```

La diferencia en este caso es que en la función división, al no acceder a los parámetros por su clave-valor, hay que añadir los dos primeros parámetros que no se utilizarán.

El uso de *args y **kwargs es delicado y se recomienda nombrar de forma explícita los parámetros a usar en cada función y dejar la potencia de su uso para los conceptos de clases y herencia que veremos más adelante.

## 6.4. Funciones recursivas

La recursividad es una técnica de programación muy útil y comúnmente utilizada en la industria del desarrollo de software. Se basa en definir funciones que se llaman a sí mismas para completar su lógica en lugar de utilizar bucles o iteraciones, aunque toda definición recursiva se podría expresar de forma iterativa.

En su estructura, las funciones recursivas presentan un patrón simple: un **caso base** por el que se terminaría o comenzaría la ejecución y **la llamada o llamadas recurrentes.**

Existen situaciones en las que el empleo de esta técnica es muy beneficioso, dado que mejora la legibilidad del código y queda patente su intencionalidad.

Ejemplo:

pruebas.py > suma_digitos

```python
def suma_digitos(numero):

    #Calcula la suma de los dígitos de un número utilizando recursividad.

    # Caso base: Si el número tiene un solo dígito, devolverlo.
    if numero < 10:
        return numero
    else:
        # Caso recursivo: Separar el último dígito y sumar con el resto
        # realizando una llamada recursiva.

        return (numero % 10) + suma_digitos(numero // 10)

# Prueba de la función
numero = 12345
print(f"La suma de los dígitos de {numero} es: {suma_digitos(numero)}")
```

Se puede resumir la ejecución recursiva de la siguiente forma:

suma_digitos(12345) = 5 + suma_digitos(1234)

suma_digitos(1234) = 4 + suma_digitos(123)

suma_digitos(123) = 3 + suma_digitos(12)

suma_digitos(12) = 2 + suma_digitos(1)

suma_digitos(1) = 1

De manera que la salida por pantalla una vez ejecutado el programa será: **15**

Otro ejemplo con la secuencia de Fibonacci de forma recursiva:

Se pretende calcular el n-ésimo término de la secuencia de Fibonacci:

pruebas.py > ...

```python
def fibo_recursivo(n):
    if n <= 1:
        return n # Caso base
    else:
        return fibo_recursivo(n - 1) + fibo_recursivo(n - 2) # Llamadas recursivas.

#ejecucion
print (fibo_recursivo(6))
```

Si pasamos como parámetro el valor 6, el término de la secuencia que se encuentra en esa posición es: **8**

Ejercicio: para practicar, realiza una función que, a partir de una cadena de texto pasada por parámetro devuelva su invertida utilizando recursividad.

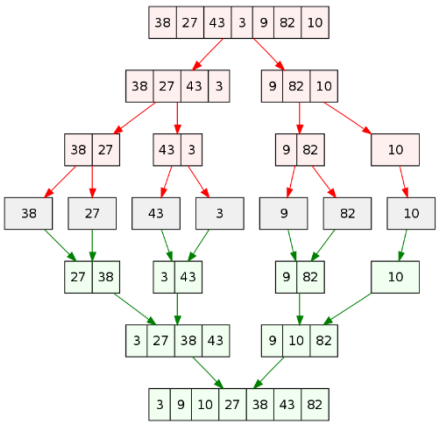
Entrada: melocotón

Salida: nótocolem

---

Veamos un ejemplo algo más “completito”:

Se pretende crear un algoritmo de ordenación que separe los valores de una lista en dos sublistas. Los valores de la primera lista deberán ser todos menores que los de la segunda. Aplique el mismo algoritmo de forma recursiva para las sublistas (este algoritmo es el conocido como merge sort)




Los pasos de este algoritmo son:

- Dividir la lista inicial hasta llegar a los elementos únicos.
- Comparar cada grupo de dos o más elementos para saber cuál es más pequeño
- Ir agrupando grupos formados anteriormente y mezclándolos de manera que solo haga falta mirar los grupos posición a posición una vez

pruebas.py > ...

```python
def merge_sort(lista):
    if len(lista)> 1:
        # calculamos el índice de la posición central
        centro = len(lista) // 2
        mitad_izq = lista[:centro] # lista izquierda
        mitad_der = lista[centro:] #lista derecha
        merge_sort(mitad_izq) # orden recursiva con la izquierda
        merge_sort(mitad_der) # orden recursiva con la derecha

        i = 0 # indice de la lista izquierda
        d = 0 # índice de la lista derecha
        x = 0 # indice de la lista original

        # Se actualiza la lista original con los menores
        # valores de cada lista dividida
        while i < len(mitad_izq) and d < len(mitad_der):
            if mitad_izq[i] < mitad_der[d]:
                lista[x] = mitad_izq[i]
                i += 1
            else:
                lista[x] = mitad_der[d]
                d += 1
            x += 1

        # si quedan elementos en izquierda o derecha
        # se añaden a la lista original

        while i < len(mitad_izq):
           lista[x] = mitad_izq[i]
           i += 1
           x += 1
        while d < len(mitad_der):
            lista[x] = mitad_der[d]
            d += 1
            x += 1

a=[3,5,234,26,23,368,2,9,78,4,128,65,20,4,789,2256]
merge_sort(a)
print(a)
```

Salida:

```
[2, 3, 4, 4, 5, 9, 20, 23, 26, 65, 78, 128, 234, 368, 789, 2256]
```



Haz el cambio necesario para que la lista de ordene de mayor a menor…

## 6.5. Funciones anónimas: Expresiones Lambda

En Python existe un tipo de expresiones especiales que permite crear funciones anónimas denominadas **expresiones lambda**. Estas funciones pueden ser utilizadas como argumentos para otras funciones o almacenadas en simples variables. Lo que ocurre es que, al no tener nombre, no pueden ser llamadas de nuevo más adelante en la ejecución salvo que se asigne a una variable.

expresión_lambda = **lambda argumentos**: **expresión**

Donde **lambda** es la palabra clave que indica el inicio de una función anónima.

**argumentos:** lista de uno o más argumentos separados por comas (similar a los parámetros de una función normal)

**expresión:** es una única expresión que se evalúa y devuelve un resultado y que puede llevar condiciones.

pruebas.py > ...

```
1    suma = lambda x,y: x + y
2    print(suma(3,4))
```

Estas funciones anónimas son utilizadas como funciones de ordenación, manipulación o filtrado en funciones como sort, sorted, min, max, map o filter, entre otras:

pruebas.py > ...

```
1    lista = ['a1','a12','a100','a5']
2    print(sorted(lista))
```

Los resultados están ordenados por carácter.

pruebas.py > ...

```
1    lista = ['a1','a12','a100','a5']
2    print(sorted(lista,key=lambda x:int(x[1:])))
```

Ahora, con la utilización de la función lambda, hemos ordenado los resultados por números enteros a partir del segundo carácter de cada valor (cadena):

Salida:

```
['a1', 'a5', 'a12', 'a100']
```

El siguiente ejemplo ordena una lista por el tamaño de cada cadena de la lista:

pruebas.py > ...
```
1   lista = ['manzana','pera','melocotón','fresa']
2   lista.sort(key=lambda x: len(x))
3   print(lista)
```

Salida:

```
['pera', 'fresa', 'manzana', 'melocotón']
```

El siguiente ejemplo devuelve e valor de la lista que tiene el mínimo “resto” al dividirlo entre 10:

pruebas.py > ...
```
1   lista = [5,425,34,7]
2   print(min(lista,key=lambda x:x%10))
```

La salida es: 34

El siguiente ejemplo nos sirve para crear una lista con los número pares entre 0 y 20 (no incluido)

```
1   lista = list(filter(lambda x: not x % 2, range(20)))
2   print(lista)
```

Salida:

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

El siguiente ejemplo genera una lista con los números cuadrados del rango de números entre el 1 y el 5:

pruebas.py > ...
```
1   resultado = map(lambda x: x**2, range(1,6))
2
3   print(list(resultado))
```

Salida:

```
[1, 4, 9, 16, 25]
```

Es muy interesante el uso de expresiones lambda para crear **funciones parciales**, denominadas así porque tienen parámetros asignados a valores y otros a la espera de ser instanciados cuando se llame a la función. Veamos el siguiente ejemplo:

pruebas.py > ...

```python
def f_principal(n):
    return lambda x: x * n # Esto devuelve
    # una función anónima donde sólo el parámetro
    # 'n' está fijado

# Realizamos las llamadas.
anonima = f_principal(9) # 'n' se ha instanciado con
                         # valor 9

resultado=anonima(10) # con esta llamada, 'x' toma el valor 10
                      # y guardamos el cálculo en resultado
print (resultado)
```

Salida: 90

Como se está viendo, las funciones **lambda** son útiles para casos puntuales, pero se recomienda el uso de funciones normales siempre que sea posible.

## 6.6. Funciones de orden superior

En la programación funcional existe el concepto de funciones de orden superior. Son funciones que aceptan otras funciones como parámetros. Python las soporta sin problemas.

Veámoslo con un ejemplo:

pruebas.py > ...

```python
import math # biblioteca que nos permite utilizar
           # funciones matemáticas

def raiz_de_par(x):
    if not x % 2:
        return math.sqrt(x) # calcula la raiz cuadrada de x
    return 0
def mul_de_par(x, multi=5):
    if not x % 2:
        return x * multi # multiplica x por 5
    return 0

def aplicar_func(lista,func):
    return [func(x) for x in lista] # Esta función
                # devuelve la función pasada por parámetro
                # con cada uno de los valores de la lista
                # como parámetro

# Realizamos las llamadas
list=[3,4,5,6]
print(aplicar_func(list,mul_de_par))
list=[25,100,81,36]
print(aplicar_func(list,raiz_de_par))
```

Salida:

```
[0, 20, 0, 30]
[0, 10.0, 0, 6.0]
```

## 6.7. Funciones dentro de funciones

Python permite que una función devuelva otra función, pero también permite definir funciones dentro de otras funciones. Estas crean un contexto propio con sus propias variables locales, lo que se conoce como **clausuras** o cerramientos. Esto es lo que se utiliza en la programación orientada a objetos para hacer encapsulamiento de funciones.

pruebas.py > ...

```python
def telefono_con_prefijo(prefijo_pais):
    def f_telefono(numero_tlf):
        return f'+{prefijo_pais} {numero_tlf}'
    return f_telefono

# Llamada a las funciones
# Definimos variables para los prefijos de los distintos paises
telefono_españa=telefono_con_prefijo(34)
telefono_aleman=telefono_con_prefijo(96)
telefono_andorra=telefono_con_prefijo(376)

# Llamamos a la función según el prefijo
print(telefono_españa(927535656))
print(telefono_aleman(256256256))
print(telefono_andorra(558585897))
```

Salida:

```
+34 927535656
+96 256256256
+376 558585897
```

Como se puede ver en el ejemplo, la función *telefono_con_prefijo* sirve para generar funciones con prefijos diferentes que se pueden reutilizar como funciones nuevas, aunque hayan sido generadas dinámicamente.

Además, se pueden crear funciones dentro de funciones de manera que se encapsule la lógica que sólo haga referencia a la lógica de la función principal, permitiendo la encapsulación en funciones más simples, que comparten las variables de la función principal:

pruebas.py > ...

```python
def beneficios(gastos:list, ingresos:list, tramos_impuestos):
    def obtener_impuesto(ingreso):
        impuesto_actual = tramos_impuestos[0][1]
        for hasta_valor, impuesto in tramos_impuestos:
            if hasta_valor > ingreso:
                break
            else:
                impuesto_actual = impuesto
        return impuesto_actual

    def calculo_neto(ingreso_bruto):
        impuesto = obtener_impuesto(ingreso_bruto)
        return ingreso_bruto - (ingreso_bruto * impuesto)

    gastos_totales = sum(gastos)
    ingresos_netos = sum(map(calculo_neto, ingresos))
    return ingresos_netos - gastos_totales

gastos = [22,314,32,52]
ingresos = [45, 623, 12, 90]
tramos_impuestos = [(20,0.06), (50,0.08), (200,0.1), (500,0.2), (float('Inf'),0.21)]
balance_final=beneficios(gastos,ingresos,tramos_impuestos)
print(balance_final)
```

El ejemplo anterior pretende crear una función que calcula las ganancias obtenidas para unos datos ficticios. La ganancia neta se calcula en función del ingreso obtenido, restándole los impuesto sobre esos ingresos, que son definidos según unos tramos de aplicación.

Como se puede ver la lógica del programa queda descrita en solo tres líneas de código y hace uso de las funciones internas definidas, que usan los argumentos de la función principal y permiten que la lógica pueda ser leída con facilidad.

## 6.8. Espacios de nombres y contextos

Es importante entender correctamente el espacio de nombres y los contextos de las variables. Esto permitirá comprender qué variables pueden ser modificadas y accedidas y cuáles no,

En Python las variables que se definen  a nivel de fichero de texto o de módulo se denominan **variable globales**, entendiendo un módulo o fichero de texto como el fichero con extensión ".py" que contiene código Python.

Las variables que se definen dentro de una función son **variables locales**. Si una función intenta leer el valor de una variable global, no tendrá problema si ha sido definida con anterioridad, pero si intenta modificar el valor de la misma se elevará una excepción de tipo *NameError* o *UnBoundLocalError,* indicando que la variable que se quiere actualizar no está definida en ese contexto.

Ejemplo:

pruebas.py > ...
```
var_global = 10
def foo(x):
    return x + var_global #El uso de esta variable está permitido

#hacemos la llamada
print(foo(5))
```

Si vemos el código anterior, estamos definiendo una variable global dentro de un programa (fichero “pruebas.py”), es decir, a nivel de módulo. Si intentamos acceder a la variable desde la función “foo” no existe problema. La salida del programa nos daría: **15**

pruebas.py > ...
```
var_global = 10
def foo(x):

    var_global += 10 # Intentamos modifaca el valor de la
                     # variable global desde la función

    return x + var_global #El uso de esta variable está permitido

#hacemos la llamada
print(foo(5))
```

Si modificamos el programa e intentamos cambiar el valor de la variable global “var_global”, comprobaremos que nos devuelve una excepción, en este caso UnBoundLocalError al estar intentando incrementarla.

Para poder modificar esa variable global es necesario hacer uso de la palabra reservada **global**, que define que la variable a utilizar pertenece al contexto superior y puede ser modificada, de lo contrario se consideraría como variable local y la modificación no se haría sobre la variable global sino que sería como tener una nueva variable dentro de la función.

pruebas.py > ...
```
var_global = 10
def foo(x):

    global var_global
    var_global += 10 # Ahora podemos modificar el valor
                     # de la variable global
    return var_global + x

#hacemos la llamada
print(var_global)
print(foo(5))
print(var_global)
```

Observa que al ejecutar el programa, ahora no se produce el error. Sacamos por pantalla el valor de la variable global antes de la llamada y modificación de la función y después de la llamada para comprobar que efectivamente se produce la modificación de su valor dentro de la función **foo().**

Del mismo modo, si una función interna es definida dentro de una función se vuelven a crear dos contextos diferentes: el contexto de la función principal y la función interna. El comportamiento es muy similar al visto anteriormente con las variables globales y locales.

En el caso de las variables de una función interna es necesario utilizar la palabra reservada **nonlocal** en lugar de **global.**

pruebas.py > ...

```
1   def foo_correct():
2
3       ruedas = 4
4
5       def pinchar_rueda():
6           nonlocal ruedas
7           ruedas -= 1
8       print (f'Número de ruedas: {ruedas}')
9       pinchar_rueda()
10      print (f'Número de ruedas: {ruedas}')
11
12
13  #Hacemos la llamada
14  foo_correct()
```

Salida:

```
Número de ruedas: 4
Número de ruedas: 3
```

## 6.9. Decoradores

Básicamente son funciones que sirven para añadir funcionalidades a otras funciones. Son funciones que reciben otras funciones (funciones de orden superior), que definen lógica interna para satisfacer una necesidad específica y manejan los parámetros arbitrarios que se pasan a esta función pasándolos a la función final (uso de *args y **kwargs).

Un ejemplo simple puede ser el siguiente, en el que se añade la funcionalidad (se decora) de el tiempo de ejecución y el nombre de la función ejecutada, para lo que vamos a crear una función decoradora que posteriormente se utilizará para decorar (añadir esa funcionalidad) a la función final, en este caso una función que transforma una cadena que recibe por parámetro:

pruebas.py > ...

```python
import time

def temporiza_funcion(funcion_final):
    def funcion_interna(*args, **kwargs):
        inicio = time.time() #devuelve la hora actual
        resultado = funcion_final(*args,**kwargs)
        tiempo = time.time() - inicio # obtenemos el tiempo que ha tardado en ejecutarse
        print(f'Funcion {funcion_final.__name__} tardó {tiempo}s. usando {args} y {kwargs}')
        return resultado
    return funcion_interna

#Declaramos la función decoradora que decorará la funcion final
@temporiza_funcion
def convierte_formato(cadena):
    lista_elems = list(cadena)
    lista_final = []
    for c in lista_elems:
        elem = c.lower() if c.lower()>'p' else c.upper()
        lista_final.append(elem)
    print (''.join(lista_final))

#Hacemos la llamada a la función que ha sido decorada
convierte_formato("Hola Mario")
```

Al ejecutar el programa anterior, obtenemos la salida:

```
HOLA MArIO
Funcion convierte_formato tardó 0.0010011196136474 61s. usando ('Hola Mario',) y {}
```

Como podemos comprobar , ejecuta la funcionalidad de la función final **convierte_formato** y al mismo tiempo es decorada con la funcionalidad de **temporiza_funcion**

Por supuesto, imagina que ahora añadimos una nueva función en nuestro programa y queremos *decorarla* con @temporiza_funcion, sólo tendremos que declarar el decorador delante de la definición de la nueva función:

```python
@temporiza_funcion
def suma(n1,n2):

    print (n1 + n2)

suma(4,2)
```

Salida:

```
6
Funcion suma tardó 0.0010116100311279297s. usando (4, 2) y {}
```



La librería estándar de Python cuenta con “decoradores” incluidos tales como: @staticmethod, @classmethod, @property, @functools.lru_cache, @functools.wraps, @dataclass, @abstractmethod.

## 6.10. Documentación de funciones

En Python, las funciones cuenta con un sistema para documentarlas. Consiste en añadir una cadena de caracteres en la primera línea de la definición. Esta cadena puede ser de una única línea o de múltiples líneas y contiene la descripción de la lógica de la función. Puede contener también los parámetros y el objeto que devuelve.

El siguiente es un ejemplo bastante completo de cómo documentar una función:

ejemplo_documentada.py > ...

```python
def calcular_estadisticas(lista_numeros):
    """Calcula estadísticas básicas (mínimo, máximo, promedio) de una lista de números.

    Argumentos:
        :lista_numeros (list): Una lista de números enteros o decimales.

    Returns:
        dict: Un diccionario con las claves:
            - "min": Valor mínimo de la lista.
            - "max": Valor máximo de la lista.
            - "promedio": Promedio de los números de la lista.

    Excepciones:
        ValueError: Si la lista está vacía.
        TypeError: Si algún elemento de la lista no es un número.

    Ejemplo:
        >>> calcular_estadisticas([10, 20, 30])
        {'min': 10, 'max': 30, 'promedio': 20.0}

        >>> calcular_estadisticas([-5, 0, 5, 10])
        {'min': -5, 'max': 10, 'promedio': 2.5} """

    if not lista_numeros:
        raise ValueError("La lista está vacía. Por favor, proporcione al menos un número.")

    if not all(isinstance(n, (int, float)) for n in lista_numeros):
        raise TypeError("Todos los elementos de la lista deben ser números enteros o decimales.")

    minimo = min(lista_numeros)
    maximo = max(lista_numeros)
    promedio = sum(lista_numeros) / len(lista_numeros)

    return {"min": minimo, "max": maximo, "promedio": promedio}


# Ejecución del ejemplo
datos = [10, 20, 30]
estadisticas = calcular_estadisticas(datos)
print(f"Estadísticas: {estadisticas}")
# La siguiente llamada devuelve la ayuda (documentación) de la función pasada por parámetro.
help(calcular_estadisticas)
```

Internamente, la documentación de las funciones se guarda en formato plano dentro de __doc__ y se puede consultar fácilmente al igual que con **help**:

**print(calcular_estadisticas.__doc_**

## 6.11. Funciones generadoras

Hemos estado viendo en apartados anteriores que Python soporta objetos generadores, que son capaces de generar secuencias de datos muy grandes con un espacio de memoria mínimo. Además, Python cuenta con las funciones generadoras que son capaces de crear generadores según las necesidades de cada caso.

Una función generadora es similar a una función 'normal" solo que en lugar de utilizar la palabra reservada **return**, se utiliza la palabra reservada **yield** cuando se devuelve el valor de la función.

Ejemplo:

func_generadora.py > ...

```python
def generar_pares(limite):
    """
    Generador que produce números pares desde 0 hasta el número limite dado (inclusive).

    Argumentos:
        limite (int): El valor máximo hasta el cual se generarán números pares.

    Yields:
        int: Números pares desde 0 hasta el máximo.
    """
    numero = 0
    while numero <= limite:
        yield numero
        numero += 2  # Incremento de 2 para generar solo números pares


# Uso del generador
for par in generar_pares(10):
    print(par)
```

Fíjate cómo tratamos el resultado que devuelve la función, como un generador.

Otro ejemplo, en este caso vamos a implementar una función generadora que simula a la función **range():**

func_generadora.py > ...

```python
def range_personal(inicio, fin, paso=1):
    salida = inicio
    while salida < fin:
        yield salida
        salida += paso

#Realizamos la llamada para probar
resultado = list(range_personal(0,20,2))
print(resultado)
```

Salida:

```
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

Observa el siguiente ejemplo:

func_generadora.py > ...

```python
def dame_nombres(*nombres):
    for nombre in nombres:
        for letras in nombre:
            yield letras

letras_nombre=dame_nombres('Antonio','Milagros','Marta','Juan')
print(next(letras_nombre))
print(next(letras_nombre))
print(next(letras_nombre))
print(next(letras_nombre))
print(next(letras_nombre))
print(next(letras_nombre))
print(next(letras_nombre))
```

Salida:

```
A
n
t
o
n
i
o
```

Lo que hacemos es crear una función generadora que recibe por parámetro un número indeterminado de nombres. Por cada nombre recorre sus letras y las devuelve con **yield**. Es decir, lo que hemos creado es un **subgenerador**. Podemos simplificar esto utilizando la sentencia **yield from:**

func_generadora.py > ...

```python
def dame_nombres(*nombres):
    for nombre in nombres:
            #eliminamos el for anterior
            yield from nombre

letras_nombre=dame_nombres('Antonio','Milagros','Marta','Juan')
print(next(letras_nombre))
print(next(letras_nombre))
print(next(letras_nombre))
print(next(letras_nombre))
print(next(letras_nombre))
print(next(letras_nombre))
print(next(letras_nombre))
```

Salida:

```
A
n
t
o
n
i
o
```

Teniendo en cuenta esto, observa el siguiente ejemplo en que utilizamos la sentencia **yield from** con una función generadora. Recuerda que una función generadora devuelve un generador, por tanto, podemos utilizar esta expresión para tratar con la función generadora.

pruebas.py > ...

```python
def generador_numeros(n):
    for x in range(n):
        yield x

def listas_de_numeros(num):
    for n_elemens in range(num):
        a = yield from generador_numeros(n_elemens)
        print(f'Enviada lista de {n_elemens} números')

numeros = listas_de_numeros(7)
# Realizamos el siguiente bucle para obtener todos los datos
# del generador numeros
while True:
    try:
        print(next(numeros))
    except StopIteration:
        break
```

Pruébala y observa el resultado.

## 7. Excepciones

Prácticamente todos los lenguajes de programación tienen herramientas que les permiten manejar problemas inesperados. Es lo que se conoce como sistema de **excepciones del lenguaje**.

Por supuesto, Python cuenta con uno de estos sistemas y además, muy extenso.

En Python, las excepciones se elevan de forma fortuita al producirse una situación inesperada o de forma voluntaria mediante la sentencia **raise.** En cualquiera de sus formas, la ejecución del programa se detendrá si no se maneja de forma correcta.

Para manejarlas, Python utiliza las sentencias **try** y **except.** Al envolver el código con estas sentencias conseguimos poder manejar la ejecución del programa y en caso de que se produzca una excepción, establecer cómo debería comportarse el programa. Es importante tener en cuenta que cuando se eleva una excepción, ésta será elevada por todas las funciones hasta llegar al contexto de la ejecución principal.

Veamos un ejemplo:

excepciones.py > ...

```python
def busca_elemento(objeto, indice_o_clave):

    """Envolvemos el código en las sentencias try except
    para manejar las posibles excepciones que se produzcan.
    En este caso vamos a utilizar las excepciones del sistema
    IndexError, KeyError y Exception (general) que estudiaremos
    más adelante"""
    try:
        #devolvemos el objeto del índice o clave
        return objeto[indice_o_clave]
    # si se ha producido una excepción al no
    # existir el objeto en el índice o clave indicados...
    except IndexError:
        print(f'índice "{indice_o_clave}" utilizado no accesible ')
    except KeyError:
        print(f'Clave "{indice_o_clave}" utilizada no encontrada')
    except Exception as e: #Cualquier otro tipo de excepción
        print(f'Excepción inesperada "{e}"')
        return -1

#inicializamos una lista
lista = [1,2,3,4]
print(busca_elemento(lista, 2))
print(busca_elemento(lista,8))

#inicializamos un diccionario
diccionario=dict(color='Verde',tipo='Coche')
print(busca_elemento(diccionario,'tipo'))
print(busca_elemento(diccionario,'marca'))
```

Si observamos la ejecución del programa, en el momento que intentamos obtener elementos de la lista o el diccionario, que no existen, se eleva una excepción que es controlada por nuestro programa y realiza lo que nosotros deseamos, en este caso enviar un mensaje de error. Prueba a realizar la llamada con algún valor que genere una excepción inesperada…

## 7.1. Excepciones conocidas

Las excepciones tienen una clase común que es **BaseException**, que tiene una propiedad denominada **args** y un método **with_trace-back.** El resto de excepciones heredan de esta clase y aportan información más específica.

A continuación se presenta un enlace a todas las excepciones de Python:

Excepciones Python

Vemos en detalle algunas de estas excepciones más conocidas y utilizadas.

**AttributeError**

Aparece cuando se intenta acceder a un atributo de un objeto que no está presente en el mismo o cuando una asignación falla:

```
entero=2
print(entero.upper())
```

Salida:

```
Traceback (most recent call last):
  File "f:\Mi Material 24_25\@vanza- Programación en Python\Ejemplos\excepciones.py", line 2, in <module>
    print(entero.upper())
AttributeError: 'int' object has no attribute 'upper'
```

Como se puede comprobar es fácil identificar la causa del error porque el mensaje de la excepción nos devuelve el tipo del objeto sobre el que se ha intentado llamar al atributo o método y el atributo o método inexistente en el objeto.

**ImportError**

Ocurre cuando el comando **import** tiene problemas para importar la librería o cuando **from…import** no encuentra la librería que se quiere importar. Se puede obtener esta excepción o una subclase de ella que es **ModuleNotFoundError.**

**IndexError**

Ocurre cuando se intenta acceder a una posición que está fuera del rango de posiciones de un objeto de tipo secuencia. Si el índice que se utiliza no es un ‘entero’, el error no es IndexError sino TypeError.

**KeyError**

Afecta a objetos que utilizan acceso con clave, como los diccionarios. Ocurre cuando se intenta acceder con una clave que no existe en el objeto.

**NameError**

Ocurre cuando un nombre local o global no es encontrado, de manera que se suele producir cuando una variable no ha sido inicializada o se escribe erróneamente.

**SyntaxError**

Se eleva cuando se produce un error de sintaxis en Python.

**TypeError**

Ocurre cuando se intenta aplicar una operación o una función sobre un objeto inapropiado.

Un ejemplo muy claro: abs('Hola')

```
Traceback (most recent call last):
  File "f:\Mi Material 24_25\@vanza- Programación en Python\Ejemplos\excepciones.py", line 1, in <module>
    print(abs('hola'))
TypeError: bad operand type for abs(): 'str'
```

**ValueError**

Esta excepción se eleva cuando un operador o función reciben un argumento del tipo correcto pero el valor no es adecuado.

Ejemplo: int('pepe') elevará esta excepción dado que la función **int** admite cadenas pero siempre y cuando sean numéricas, por ejemplo int('123'). Es decir, el problema no es el tipo (cadena) sino el valor.

**KeyboardInterrupt**

Es elevada cuando un usuario presiona la tecla de interrupción (Ctrl-c o Delete) durante la ejecución del programa.

**StopIteration**

Se produce cuando un iterador no tiene más elementos por devolver. Aunque se eleve siempre, el hecho de manejar el interador con bucles (for o while, por ejemplo) hace que no sea necesario manejar la excepción.

**UnboundLocalError**

Se eleva cuando se intenta hacer referencia a una variable local dentro de una función sin que el valor de la variable se haya definido:

excepciones.py > ...

```
1  def func():
2      valor_variable += 1
3      return valor_variable
4  print (func())
```

Salida:

```
Traceback (most recent call last):
  File "f:\Mi Material 24_25\@vanza- Programación en Python\Ejemplos\excepciones.py", line 4, in <module>
    print (func())
  File "f:\Mi Material 24_25\@vanza- Programación en Python\Ejemplos\excepciones.py", line 2, in func
    valor_variable += 1
UnboundLocalError: local variable 'valor_variable' referenced before assignment
```

**ZeroDivisionError**

Se eleva cuando el divisor o denominador de una división es 0.

## 7.2. Elevar excepciones de forma manual.

En Python, al igual que en otros lenguajes, las excepciones se elevan de forma automática cuando un error o acción inesperada ocurre. Pero es posible elevarlas de forma intencionada con el comando **raise** seguido de la excepción que se quiera elevar.

Lo vemos con el siguiente ejemplo:

excepciones.py > ...

```python
def comprobar_edad(edad):
    if edad < 18:
        # Lanzamos la excepción "ValueError"
        raise ValueError("La edad debe ser 18 o mayor.")
    return "Acceso permitido"

#Ejecutamos y controlamos la excepción
try:
    edad_usuario = 16
    print(comprobar_edad(edad_usuario))
except ValueError as e:
    print(f"Error: {e}")
```

Salida:

```
Error: La edad debe ser 18 o mayor.
PS F:\Mi Material 24_25\@vanza- Programación en Python\Ejemplos>
n/Ejemplos/excepciones.py"
Error: La edad debe ser 18 o mayor.
```

## 7.3. Definición de excepciones propias

Como se ha mencionado, Python permite definir una excepción basándose en una clase de excepción definida en la librería estándar o en una propia creada por nosotros mismos.

Todas las excepciones heredan de la clase **BaseException** por lo que nosotros podemos definir nuestra propia clase que herede de una de las clases del sistema. Veamos el siguiente ejemplo en el que creamos una clase de excepción que hereda de **Exception:**

excepciones.py > ...

```python
# Definición de la excepción personalizada
# que hereda de Exception
class EdadNoValidaError(Exception):
    def __init__(self, edad, mensaje="La edad proporcionada no es válida. Debe ser mayor o igual a 18."):
        self.edad = edad
        self.mensaje = mensaje
        super().__init__(self.mensaje)

# Uso de la excepción personalizada
def comprobar_edad(edad):
    if edad < 18:
        raise EdadNoValidaError(edad)
    return "Acceso permitido"

try:
    edad_usuario = 16
    print(comprobar_edad(edad_usuario))
except EdadNoValidaError as e:
    print(f"Error: {e.mensaje} (Edad proporcionada: {e.edad})")
```

Se ha creado la clase EdadNoValidaError que hereda de Exception. La clase se define con el atributo edad y un mensaje “La edad proporcionada…”

Cuando elevemos esa nueva excepción con la sentencia **raise** como ya hemos visto con anterioridad, podremos hacer uso de los atributos edad y mensaje para manejar el error producido dentro de las sentencias **try..except**. En este ejemplo aparecen elementos relativos a la programación orientada a objetos que aún no hemos visto como es la creación de la propia clase.

Veamos un nuevo ejemplo en el que lo que se pretende es crear nuevas excepciones que sean más específicas que las excepciones del sistema IndexError de manera que se especifique si el índice no ha sido encontrado en una tupla o en una lista. Las llamaremos TupleIndexError y ListIndexError:

excepciones.py > ...

```python
import sys
class TupleIndexError(IndexError):
    pass
class ListIndexError(IndexError):
    pass

def get_index_value(objeto,indice):
    try:
        return objeto[indice]
    except IndexError as e:
        args, description, tb = sys.exc_info()
        # si el objeto es una tupla...
        if isinstance(objeto,tuple):
            raise TupleIndexError(description).with_traceback(tb)
        # si no, si el objeto es una lista
        elif isinstance(objeto,list):
            raise ListIndexError(description).with_traceback(tb)
        # y si no es ninguno de los objetos
        else:
            raise e

print(get_index_value((1,2,3),4))
```

En este ejemplo, lo que se hace es crear las clases de las excepciones pero sin atributos, de manera que al lanzar las excepciones en función del objeto (tupla o lista), devolverá una u otra excepción utilizando la propia descripción y la traza del error de IndexError.

Este mecanismo ayuda mucho cuando se pretende crear una librería o un programa que tenga excepciones propias.